

Wstęp

Problemy z uczeniem

Po implementacji algorytmu, zauważyłem problem z uczeniem w `CartPole`, przy czym dla `LunarLander` wszystko działało poprawnie. Pierwotnie, przekazywaliśmy `done` do `agent.learn(state, reward, next_state, done)`, gdzie `done` to alternatywa `truncated` i `terminated`. `Truncated` oznacza przerwanie z zewnątrz, np. poprzez osiągnięcie 1000 kroków w epizodzie, `terminated` oznacza zakończenie ze strony środowiska, np. wylądowanie, czy rozbitcie.

W `compute_loss` mam linijkę: `target = r + self.gamma * (1.0 - terminal) * keras.ops.stop_gradient(v_s_next)`

Oznacza to, że gdy kończymy epizod, a patyk ciągle stoi w pionie, fałszywie mówimy, że $\text{target} = r + 0 * V(s') = r + 0 = r = 1$, przy czym gdy agent doszedł do kroku 500, to prawdziwa wartość tej obserwacji to mniej więcej $V(s_{500}) = 1 + 0.99 + 0.99^2 + \dots \approx 100$. Wmawiamy krytykowi, że wartość tej ostatniej obserwacji to dokładnie 1, czyli słabo.

Rozwiązanie to przekazywanie tylko `terminated` - czyli przyszłości nie ma, epizod się zakończył na pewno sukcesem, porażką itd.

Podsumowanie błędu wygenerowane przez AI:

Bug aktywuje się tylko gdy `truncated=True` (bo wtedy niesłusznie zerujemy $V(s')$). Jak często ta ścieżka się odpala?

`LunarLander-v3` kończy się przez:

`terminated=True` w czterech głównych przypadkach:

lądowanie (sukces)

rozbitcie

wyleciał poza ekran

całkowite zatrzymanie na ziemi (oba kontaktery dotykają podłoża)

`truncated=True` odpala się dopiero po 1000 krokach bez żadnego z powyższych - czyli gdy lądownik zawisa w powietrzu i nie podejmuje decyzji.

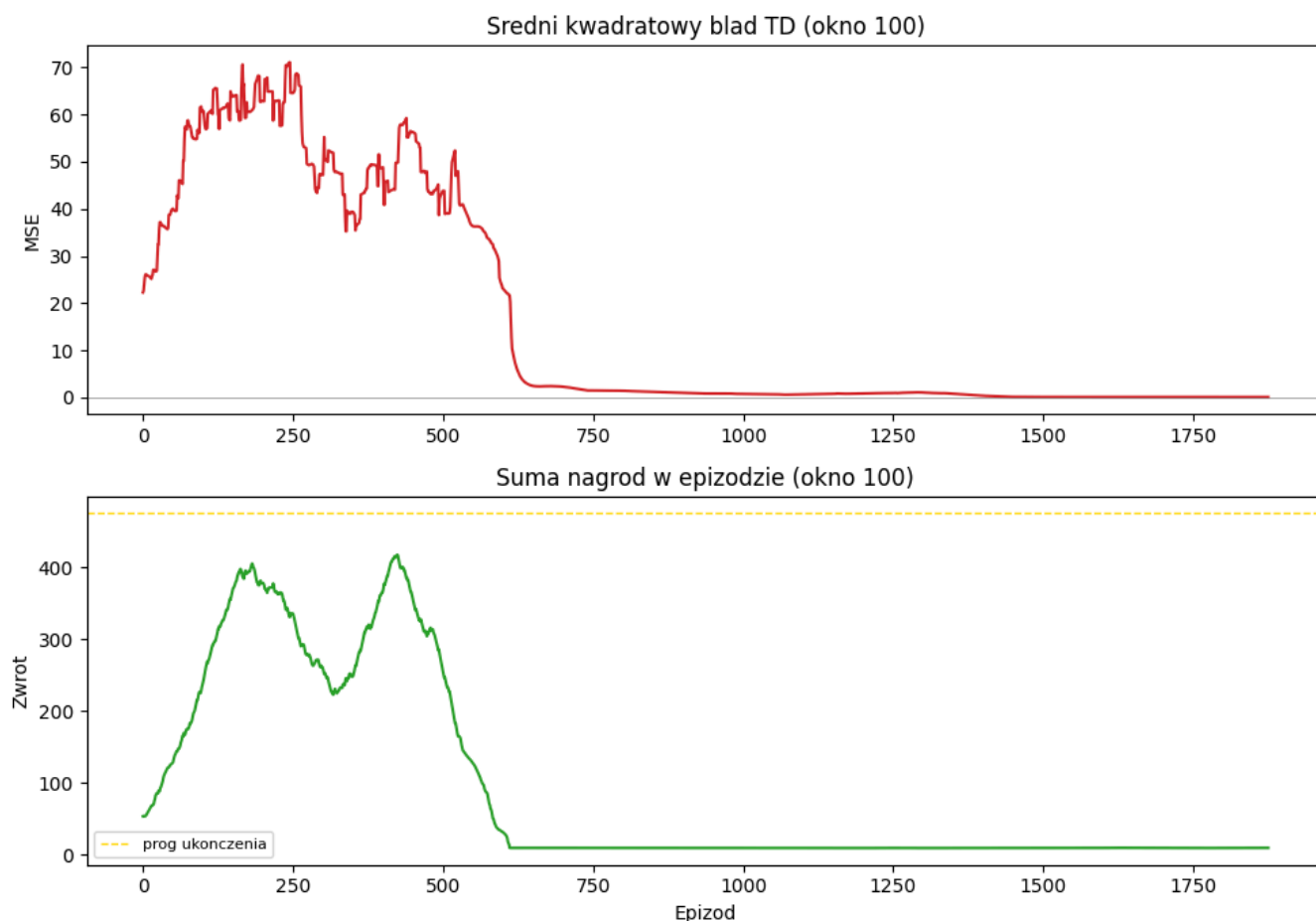
Oznacza to, że:

- Wytrenowany agent ląduje w 200-400 krokach → `terminated` → bug się nie odpala.
- Średnio wytrenowany rozbi się szybko → `terminated` → bug się nie odpala.
- Świeży agent często rozbi się w pierwszych 100-200 krokach (silniki włączone losowo, paliwo się kończy, leci w bok) → `terminated` → bug nie odpala.

Jedyna sytuacja `truncated`: agent znajdzie patologiczną politykę zawisania na silniku głównym przez 1000 kroków - to rzadkość, bo używanie silnika kosztuje (-0.3 reward/krok), więc gradient szybko od tego odciąga.

W CartPole jest dokładnie odwrotnie: najlepsza możliwa polityka prowadzi WPROST do truncation (idealny balans = 500 kroków = truncated). Bug uderza w 100% epizodów które agent rozegrał perfekcyjnie.

Załączam też zdjęcie błędów i nagród dla CartPole dla błędnego algorytmu:



Podsumowanie implementacji

Nie będę przytaczać teorii dotyczącej zaimplementowanego algorytmu, ani większych fragmentów kodu, ze względu na długość raportu. Wstawiam jedynie funkcję `compute_loss(...)`:

```
def compute_loss(self, action, x_s, x_s_next, reward, terminal):  
    """  
    Oblicza laczna strate: L = L_krytyk + L_aktor + L_entropia  
    """  
    logits_s, value_s = self.model(x_s, training=True)  
    v_s = value_s[0, 0]  
  
    # TODO: oblicz cel TD (target)  
    # Podpowiedz: nie zapomnij o keras.ops.stop_gradient...  
    _, value_s_next = self.model(x_s_next, training=False)  
    v_s_next = value_s_next[0, 0]  
  
    r = keras.ops.cast(reward, "float32")  
    target = r + self.gamma * (1.0 - terminal) * keras.ops.stop_gradient(v_s_next)
```

```
# TODO: oblicz blad TD (delta = target - V(s)) i zapisz jego kwadrat
# do self.last_error_squared (przydatne do wykresow)
# Podpowiedz: a tu się przyda keras.ops.square...
error = target - v_s # TODO: zastap poprawna formula
self.last_error_squared = float(keras.ops.square(error))

# TODO: oblicz strate krytyka jako kwadrat bledu TD
critic_loss = keras.ops.cast(keras.ops.square(error), "float32")

# TODO: oblicz log-prawdopodobienstwa akcji (log-softmax)
# Podpowiedz: log_probs = logits_s - keras.ops.logsumexp...
# Nastepnie wybierz log-prawdopodobienstwo konkretnej wybranej akcji
log_probs = logits_s - keras.ops.logsumexp(logits_s, axis=-1, keepdims=True)
log_prob_a = log_probs[0, action]

# TODO: oblicz strate aktora (policy gradient)
# Podpowiedz: tu tez sie przyda keras.ops.stop_gradient...
actor_loss = -log_prob_a * keras.ops.stop_gradient(error)

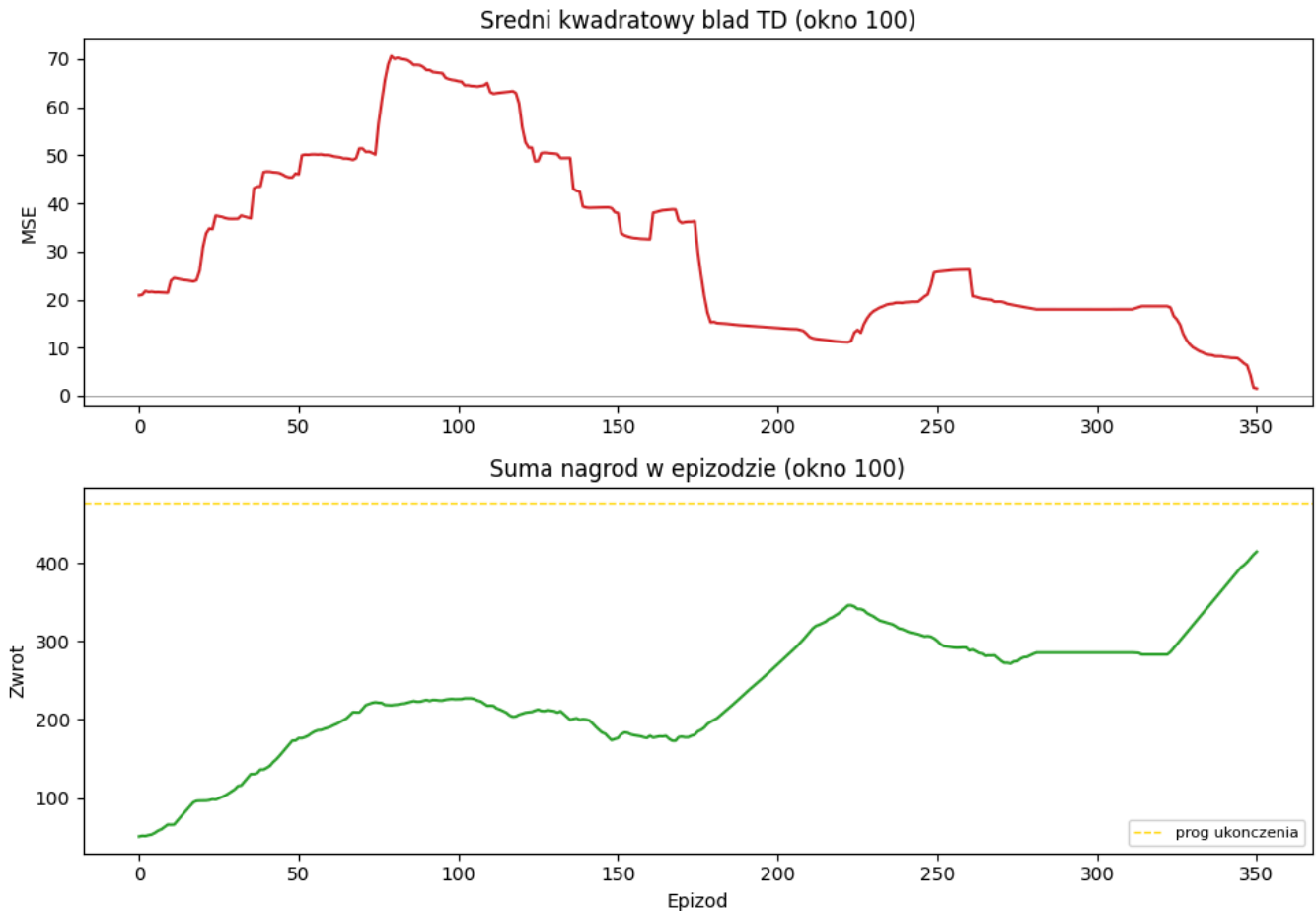
# TODO: oblicz bonus z entropii
# i mnoz przez -entropy_coeff (maksymalizacja entropii = minimalizacja -
entropii)
# Podpowiedz: log_probs zostalo juz obliczone powyzej...
probs = keras.ops.softmax(logits_s)
entropy = -keras.ops.sum(probs * log_probs)
entropy_loss = -self.entropy_coeff * entropy

return critic_loss + actor_loss + entropy_loss
```

Implementacja algorytmu przebiegła spokojnie, instrukcje zarówno w kodzie, jak i w dokumencie PDF były przejrzyste. Pułapką były momenty, w których należało wyłączyć liczenie gradientów, ale o tym też było wspomniane.

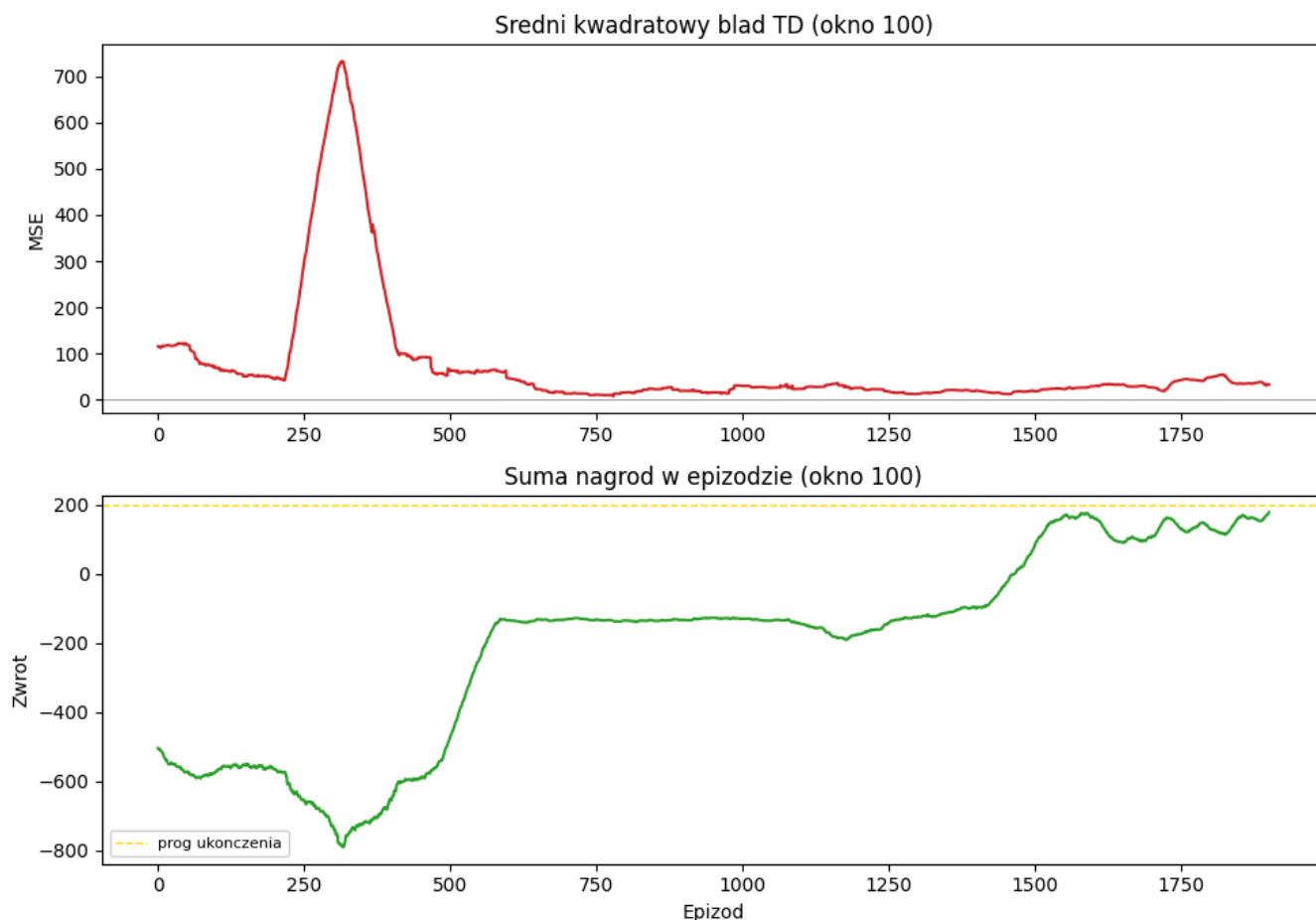
Pierwsze testy - wspólny trzon sieci neuronowej

Pierwsze uruchomienie z domyślnymi wartościami parametrów dla CartPole



Na wykresach widać zależność między błędem krytyka (MSE TD) a zwrotem agenta. Na początku (ep. 0-75) MSE rośnie, bo rosną same zwroty - większe wartości Q oznaczają większe błędy do kwadratu. Nie jest to objaw złego uczenia. W fazie stabilizacji (ep. 75-175) MSE nie spada, krytyk nie poprawia swoich oszacowań, więc aktor dostaje zaszumiony sygnał i polityka nie postępuje. Przełomy następują wtedy, gdy MSE zaczyna spadać (ok. ep. 175 i 320) - ustabilizowany krytyk daje aktorowi wiarygodny gradient i zwrot skacze w górę.

Pierwsze uruchomienie z domyślnymi wartościami parametrów dla LunarLander



Trening LunarLander przebiegał w trzech wyraźnych fazach.

Faza kryzysu (ep. 250-400): MSE wystrzeliwuje do ~ 730 , a nagroda spada do -800. Agent znalazł patologiczną politykę, której krytyk nie umiał wycenić, stąd ogromne błędy TD. Kryzys jest jednak produktywny - po nim następuje największy postęp. Stabilizacja (ep. 400-600): MSE spada z ~ 700 do ~ 50 , nagroda skacze z -700 do -130. Ten sam wzorec co w CartPole - ustabilizowany krytyk daje aktorowi wiarygodny gradient. Długie okres bez zmian (ep. 600-1400): MSE jest niski i stabilny ($\sim 20-30$), ale nagroda zatrzymuje się na ~ -130 .

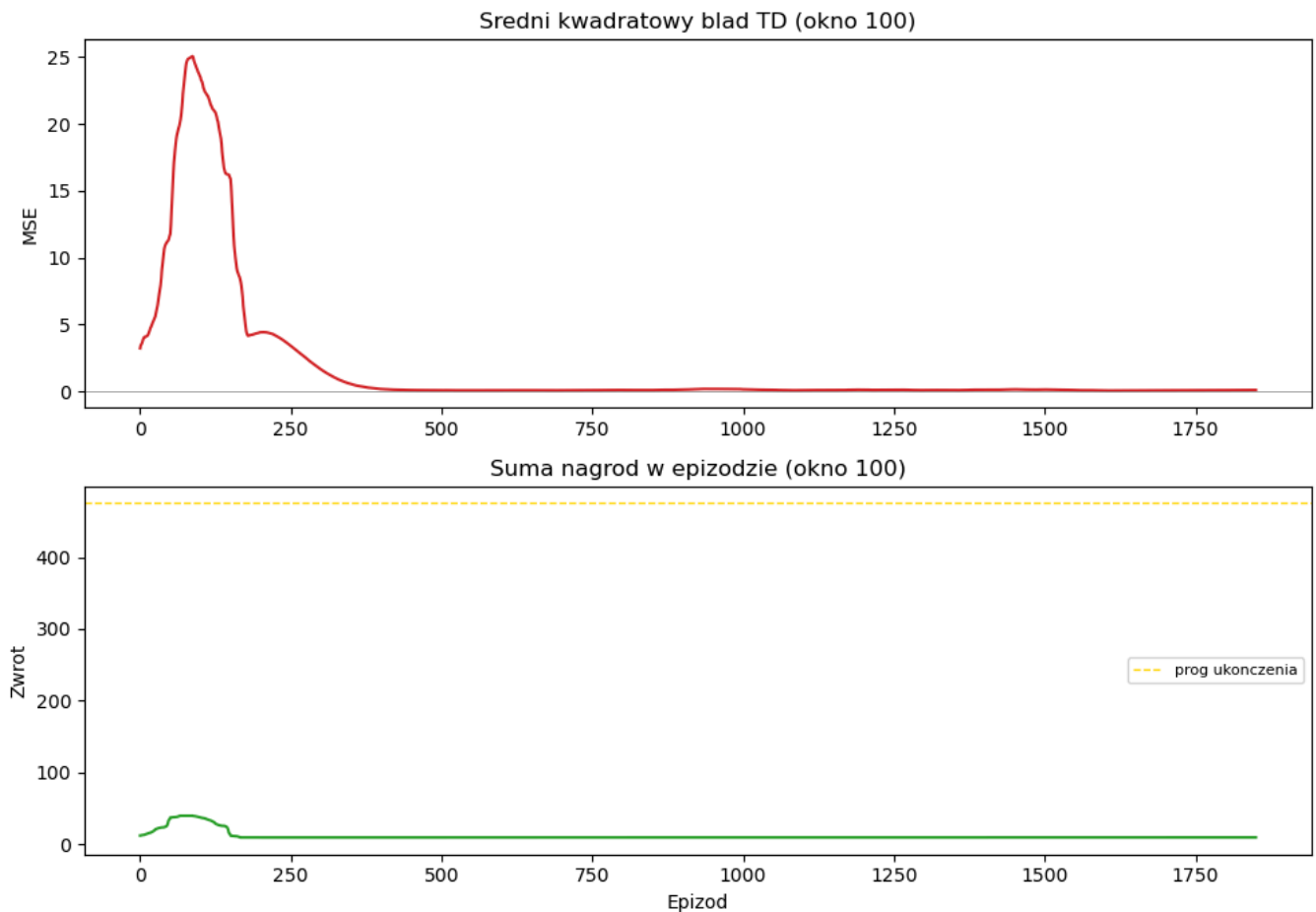
Dwie rozłączne sieci neuronowe

Wydaje mi się, że połączony trzon bardziej przeszkadza, niż pomaga, ale to wszystko zależy od środowiska. Wspólny trzon oznacza mniej parametrów, ale powoduje konflikt skali gradientów - gradient służy bardziej krytykowi, aktor praktycznie się nie uczy (widać to w LunarLander i dużym błędzie MSE u góry). Wspólny trzon jest zatem w porządku dla prostych środowisk (CartPole), ale w trudniejszych (LunarLander) przeszkadza.

Przeprowadzę teraz eksperymenty z rozłącznymi sieciami neuronowymi.

Dwie rozłączne sieci neuronowe dla CartPole

Tutaj pojawiła się ciekawa rzecz - krytyk nauczył się doskonale przewidywać porażkę, a aktor trzyma się fatalnej polityki (zapaść polityki). Polityka się nie zmienia. Wydaje mi się, że może to wynikać z dużej liczby neuronów w warstwach ukrytych. Agent przestał eksplorować, entropia jest równa 0.



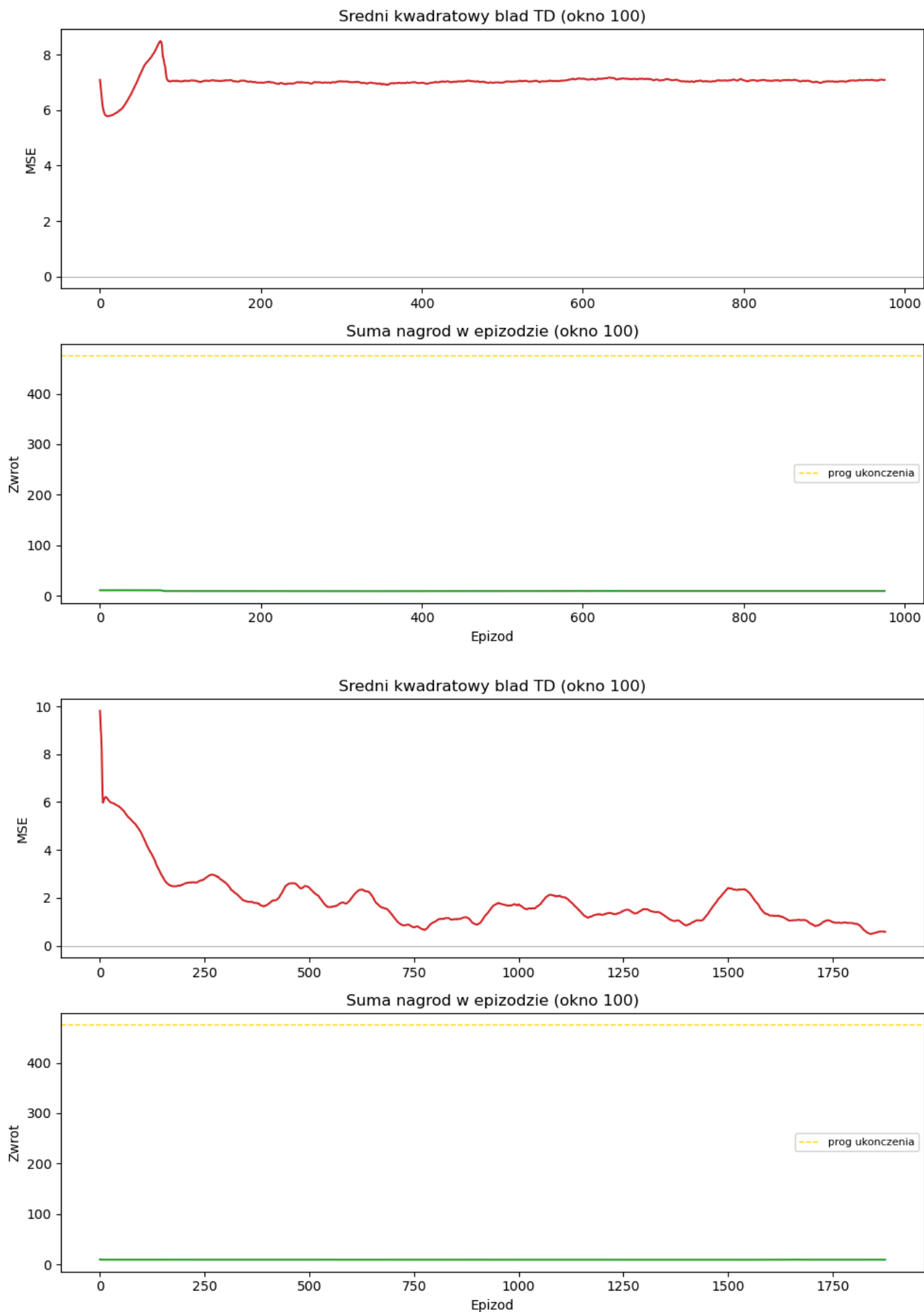
Dwie rozłączne sieci neuronowe dla LunarLander

Dla tego problemu, osobne sieci neuronowe poprawiają proces uczenia oraz rezultaty. Po pierwsze, rezultaty nauczania są znacznie szybsze - już w kroku około 600 otrzymuje zwrot na poziomie 170. Nie pojawiły się też nagłe zapaści nagrody, błędy MSE też jakoś gwałtownie nie wystrzelują (w porównaniu do pierwszego uruchomienia na samej górze raportu).



Dwie rozłączne sieci neuronowe dla CartPole z większym alpha

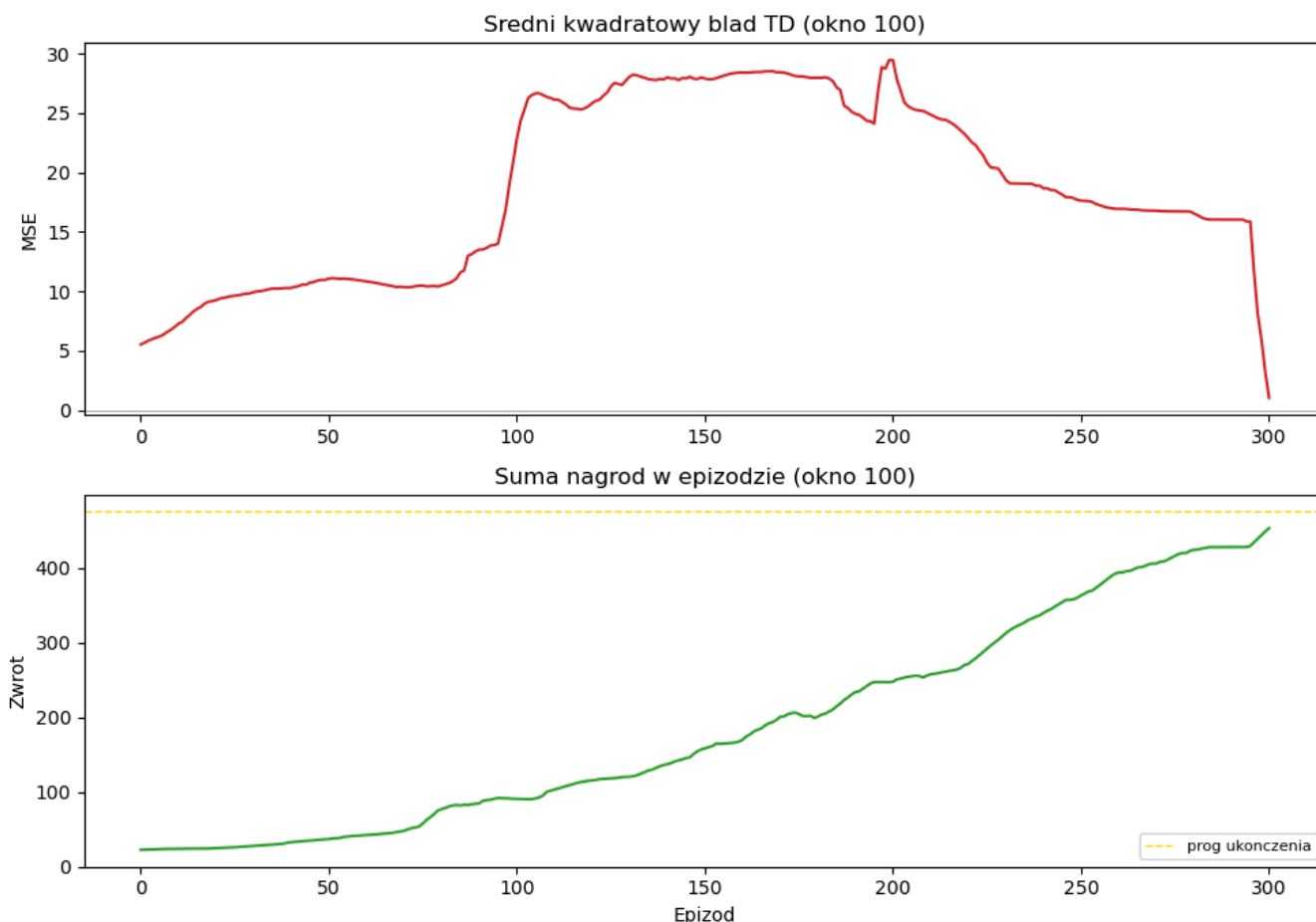
Przyjęty początkowo krok uczenia $a=1e-3$ zwiększam do $a=0.01$. W tym przypadku doszło do saturacji aktora - po określonej liczbie epizodów obie krzywe stają się martwymi liniami. Softmax się saturuje - jeden logit jest dużo większy od innych, więc polityka staje się deterministyczna, a gradient jest równy 0. Aktor traci możliwość zmiany polityki. Po wyrenderowaniu animacji, zobaczymy, że w każdy epizod będzie wyglądać tak samo.



Dwie rozłączne sieci neuronowe, mniejsza ilość neuronów w warstwach ukrytych dla CartPole

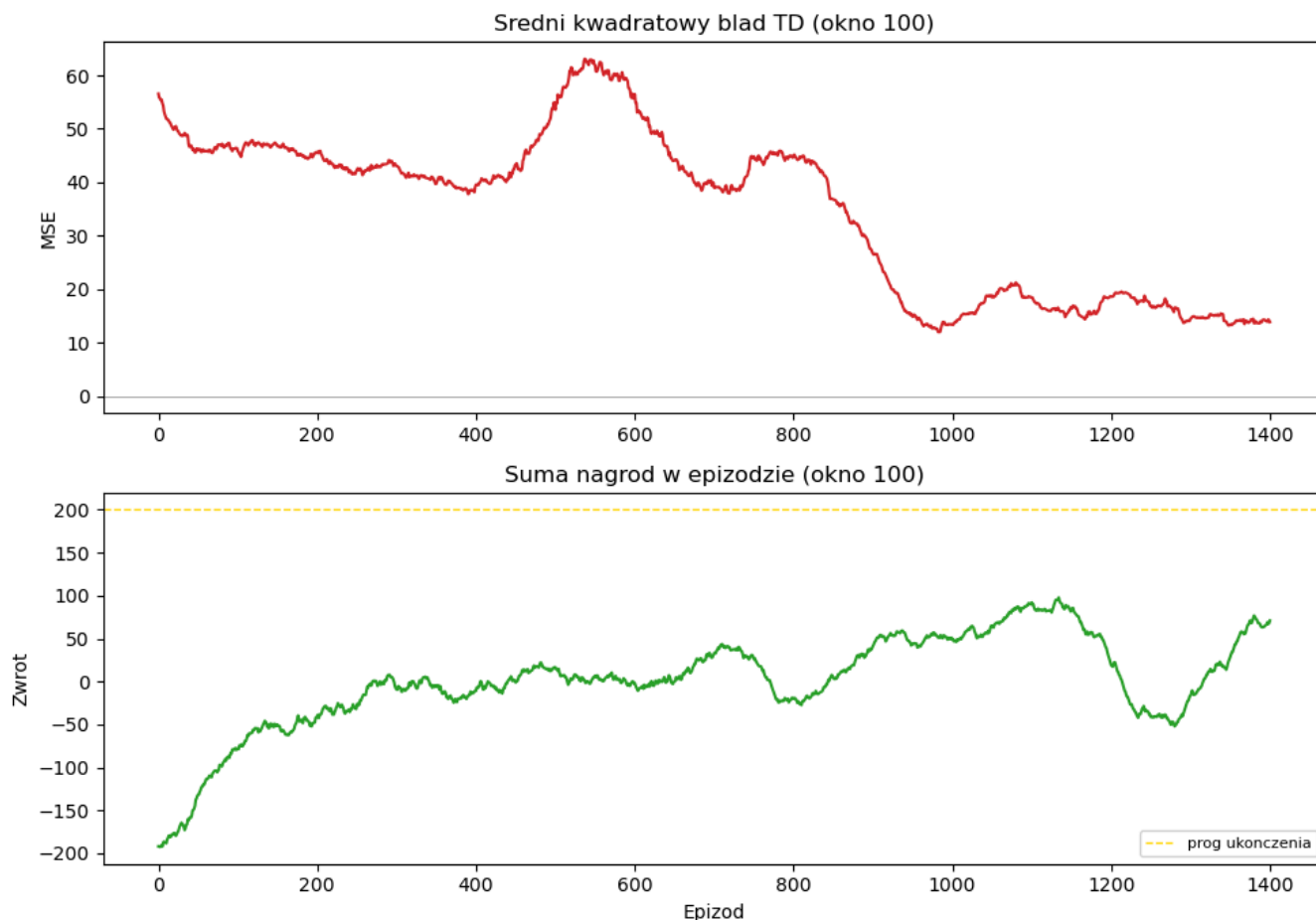
Jeden z najlepszych uruchomień CartPole. Zmniejszenie rozmiarów sieci działa na korzyść środowiska CartPole, proces uczenia jest szybszy. Patyk jest utrzymywany cały czas w pozycji pionowej przez epizod. Niemal liniowy, monotoniczny wzrost nagrody, brak oscylacji, brak kryzysów i zapaści.

Czy 128 neuronów jest konieczne? Nie. Dla CartPole pojemność (32, 32) jest nie tylko wystarczająca, ale wręcz **lepsz**a - uczenie jest szybsze (próg osiągnięty w ~300 epizodach zamiast 350+) i unika zapaści polityki obserwowanej wcześniej przy `separate=True` z 128 neuronami.



Dwie rozłączne sieci neuronowe, mniejsza ilość neuronów w warstwach ukrytych dla LunarLander

Wzrost nagród jest dość powolny, ale nie pojawiają się duże zapaści. Polityka utknęła w lokalnym minimum. Agent uczy się lądować, chociaż nie jest ono precyzyjne i szybkie. Często na chwilę zawisa w powietrzu, robi korekty.

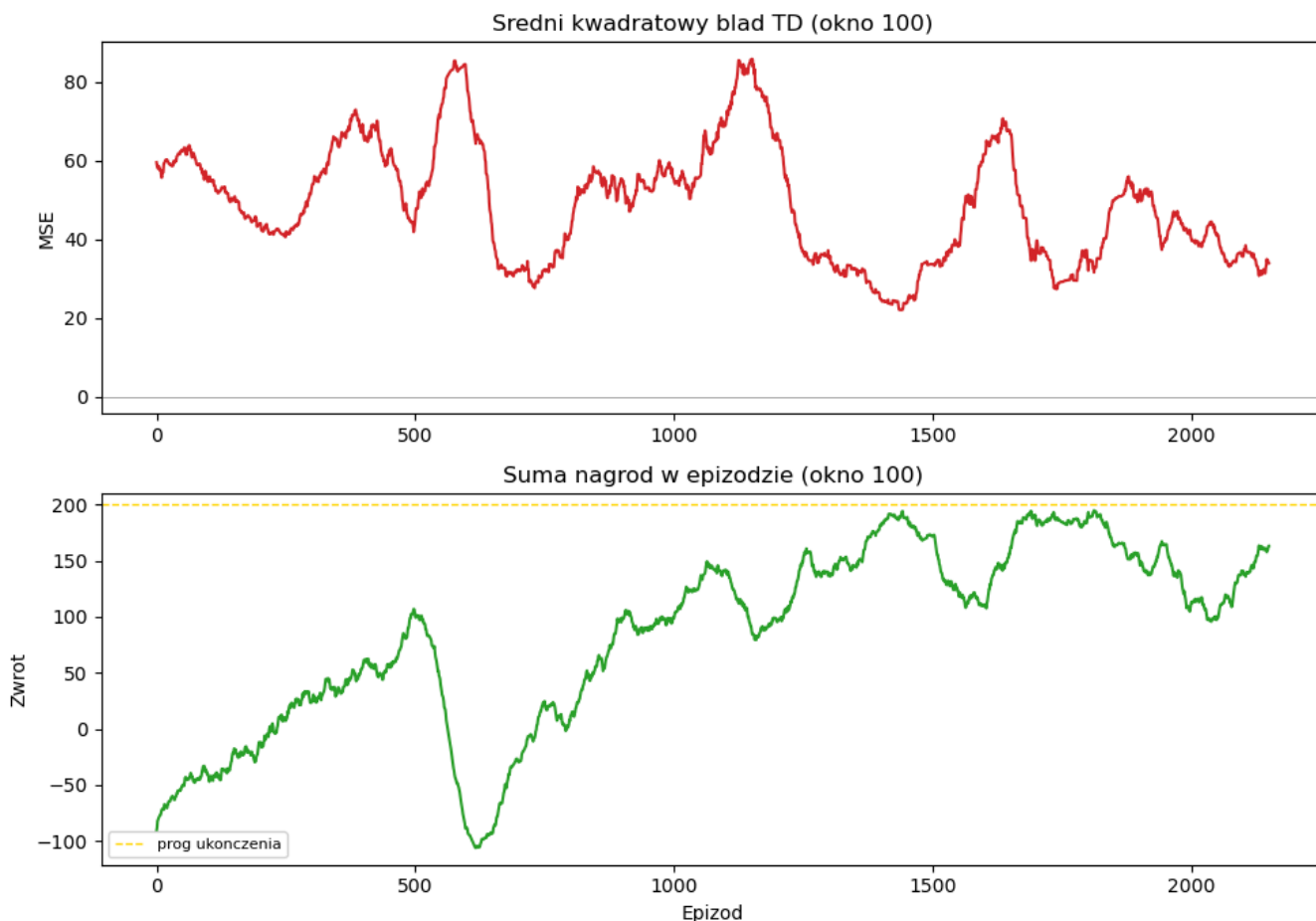


Dwie rozłączone sieci neuronowe, entropia na zero

Bez entropii LunarLander uczy się, ale niestabilnie. Pojawia się klasyczna katastrofa zapomnienia, wysokie oscylacje MSE (25–85) i niezdolność do utrzymania polityki powyżej progu mimo wielokrotnego jego dotknięcia. Entropia działa tu jako regularyzator stabilizujący - utrzymuje softmax z dala od saturacji, dzięki czemu aktor nie potrafi gwałtownie nadpisać działającej polityki nieprzetestowanym wariantem.

Czy polityka zapada się szybciej? W sensie pełnej zapaści jak w CartPole z `separate=True` - nie, agent nadal się uczy i dochodzi blisko progu. Ale występuje katastroficzna zapaść w trakcie treningu (drop +110 -> -100 w okolicy ep. 600), której w runach z $\beta=0.01$ nie obserwowaliśmy.

Czy osiąga gorsze wyniki? Tak. Mimo dotknięcia ~195 kilka razy, agent nigdy nie utrzymał średniej 100-epizodowej powyżej progu 200. Dla porównania, runy z $\beta=0.01$ osiągały gładkie ~180–190 i były bliższe ukończenia.



Podsumowanie

Przeprowadzone eksperymenty obejmowały implementację algorytmu Actor-Critic w wariantcie one-step TD oraz analizę wpływu kluczowych hiperparametrów na proces uczenia w środowiskach CartPole-v1 i LunarLander-v3. Większą wartością poznawczą miały przebiegi nieudane - pozwoliły zidentyfikować konkretne mechanizmy destabilizujące algorytm i powiązać je z wyborami architektonicznymi.

Główne wnioski

- **Rozróżnienie `terminated` od `done` w celu TD jest krytyczne.** Pomieszenie obu flag w sygnale terminalnym powoduje systematyczne zaniżenie wartości stanów końcowych, co najsilniej uderza w CartPole, gdzie ścieżka sukcesu prowadzi przez `truncated`.
- **Pojemność sieci (32, 32) była wystarczająca dla CartPole** i pozwoliła osiągnąć szybszą oraz stabilniejszą zbieżność niż konfiguracja (128, 128). Dla LunarLandera ta sama redukcja okazała się zbyt agresywna - agent uczył się polityki zwisania, ale nie precyzyjnego lądowania.
- **Zbyt wysoki `learning_rate` ($\alpha = 0.01$)** powoduje saturację softmaxu w pierwszych ~50 epizodach. Polityka staje się deterministyczna, pochodna zanika, a uczenie nieodwracalnie się zatrzymuje.
- **Entropia w LunarLanderze pełni dwie role:** eksploracyjną i regularizacyjną. Wyłączenie $\beta = 0$ nie blokuje uczenia całkowicie, ale prowadzi do katastrofalnych spadków polityki w trakcie treningu i niezdolności do utrzymania średniej powyżej progu mimo jego okazjonalnego osiągnięcia.

Rezultaty ilościowe

Środowisko	Najlepszy run	Próg	Liczba epizodów	Stan końcowy
CartPole-v1	<code>separate=True, (32, 32)</code>	475	~300	osiągnięty próg
LunarLander-v3	<code>separate=True, (256, 256), β=0.01</code>	200	~1500	~190 (nieukończony)